UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The addressing technique illustrated in Figure 2 is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective memory address of the operand, so the operand is fetched directly from memory location A.

(b) In direct addressing the instruction is held in the format [Opcode | Address A]. The CPU decodes the opcode, takes the address field A as the effective address (EA = A), and accesses main memory at that address to read the operand. The operand is then used by the operation (e.g. loaded into a register or used by an ALU instruction). No additional address calculation or pointer dereference is required: the address in the instruction points straight at the operand in memory.

(c) Advantage: it is simple and fast — only one memory reference is needed to fetch the operand (no extra indirection), and no address computation is required. Disadvantage: the address field must be large enough to hold a full memory address, which limits the range of addressable memory (and the operand address is fixed at assembly time, giving limited flexibility).

Unofficial marking guide:

• 1 mark: correctly labels the technique as direct addressing (accept absolute addressing).

• 2 marks: explains that the address field A is the effective address (EA = A) and that the operand is fetched directly from memory at that address (1 mark for EA = A, 1 mark for direct memory fetch/use of operand).

• 2 marks: 1 mark for a valid advantage (e.g. simple/only one memory access/no address calculation) and 1 mark for a valid disadvantage (e.g. limited addressable range/large address field needed/fixed address).

Model confidence: high.

Model note: Figure 2 is inferred from the extracted text: an instruction with Opcode and Address A, with Address A pointing to a memory operand. This is the standard direct-addressing diagram; if the figure shows an extra indirection (address pointing to a pointer), the answer would instead be indirect addressing.

QUESTION A2 (5 marks)

Topic: Data representation and number systems

Answer: left -10 -> 1 0001010; right 74 -> 0 1001010; -10 - 74 = -84 -> sign-magnitude 1 1010100

• 1 mark for converting both operands to sign-magnitude.

• 1 mark for applying the sign rule for subtraction.

• 1 mark for the correct magnitude operation.

• 1 mark for the correct sign bit.

• 1 mark for the final decimal and byte representation.

QUESTION A3 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (6).pdf, page(s) 7

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) One advantage of SMP applied in NUMA: all processors share a single global physical address space (single-system image / shared memory programming model), so any processor can access any memory location and programs can be written as ordinary shared-memory code, while NUMA adds a local/remote memory hierarchy to improve scalability. (Equivalently: simple shared-memory programming and coherent global memory across processors.)

(b) One disadvantage of cluster architecture: it is a loosely coupled system with no shared global memory, so inter-node communication must be done by message passing over a network. This gives high communication latency and lower bandwidth than shared-memory access, and requires complex distributed software (message passing, distributed consistency, load balancing), so programming is harder and performance can degrade when nodes exchange large amounts of data.

(c) A possible cluster architecture based on Figure 3: take the NUMA node of Figure 3 (processors + local main memories connected by an interconnection network) as one complete computer/node, then

© Hak Cipta Universiti Teknologi MARA

connect several such nodes through a high-speed network (LAN/switch) with a cluster middleware layer. Diagram description: [Node 1: CPU + local memory] --- [Node 2: CPU + local memory] --- ... --- [Node n: CPU + local memory], each node being an independent computer (optionally SMP/NUMA internally), all joined by a high-speed interconnection network/switch; each node runs its own OS and they cooperate via message passing. (In the exam, draw several boxes each containing the Figure 3 NUMA structure, linked by a network cloud/switch.)

Unofficial marking guide:

• 1 mark: (a) correct SMP advantage applied to NUMA, e.g. single shared global address space / shared-memory programming model across all processors.

• 1 mark: (b) identifying that clusters are loosely coupled with no shared memory, requiring message passing over a network.

• 1 mark: (b) explaining the consequence, e.g. high communication latency / harder distributed programming / performance loss on heavy inter-node traffic.

• 1 mark: (c) drawing multiple independent computers (nodes), each based on the Figure 3 NUMA structure (processors + local memory).

• 1 mark: (c) connecting the nodes with a high-speed network/interconnection (switch/LAN) and showing them as a cooperating cluster.

Model confidence: medium.

Model note: The extracted figure text ('tern', '2-1', 'Main') is incomplete; the answer assumes Figure 3 shows a NUMA node of processors with local main memories joined by an interconnection network. Part (c) is described in words because the actual diagram cannot be reproduced; the student should draw the described structure. 'Mufti-Processor' is read as 'Multi-Processor'.

QUESTION A4 (5 marks)

Topic: Computer generations and history

Source: CSC429 (7).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The generation described is the Fourth Generation of computers (approximately 1971–present), characterised by microprocessors, GUIs, the mouse, and the growth of computer networks leading to the Internet.

(b) The two main technologies available in this generation are: (1) Microprocessors / VLSI (Very Large Scale Integration) integrated circuits — the CPU on a single chip, enabling personal computers; and (2) Graphical User Interface (GUI) with mouse/pointing-device technology (supported by networking technology such as LANs and the Internet).

(c) Two main features of this generation are: (1) Use of microprocessors/VLSI chips giving small, cheap, reliable and portable personal computers; and (2) GUI-based, user-friendly interaction using mouse and icons, together with networking/Internet connectivity enabling resource sharing and communication.

Unofficial marking guide:

• 1 mark: (a) correctly names the Fourth Generation.

• 2 marks: (b) one mark each for microprocessor/VLSI technology and GUI/mouse (or networking) technology.

• 2 marks: (c) one mark each for any two valid features (e.g., microprocessors/VLSI, GUI/user-friendly interface, networking/Internet, personal computers, portability, low cost).

Model confidence: high.

Model note: The statement's wording ('remark the beginning') is treated as 'mark the beginning'. Some textbooks place networking/Internet mainly in the fourth generation and others emphasise it in the fifth; the answer follows the statement's own framing. Accept networking/Internet as a valid technology or feature in (b)/(c).

QUESTION A5 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (11).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The non-maskable interrupt (NMI) is message (b): "A hardware failure has been found. Please restart your computer now." Justification: an NMI is reserved for catastrophic events such as a hardware failure, power failure or memory error; it cannot be disabled or ignored by the processor (it is not maskable), so the system must respond immediately, typically by halting or restarting. Message (a), the business-class upgrade offer, is an ordinary, deferrable event generated by the web/booking application; it can be ignored or postponed by the user and is therefore a maskable interrupt.

(b) A maskable interrupt can be temporarily disabled (masked) by the processor, for example by clearing

© Hak Cipta Universiti Teknologi MARA

the interrupt-enable flag or by an interrupt mask register, so its servicing can be delayed until the CPU is ready; it is used for normal I/O and software events. A non-maskable interrupt cannot be disabled by the processor; it is always recognised and is used for urgent fault conditions such as hardware failure, so that critical events are never lost.

(c) The common way to handle multiple interrupts is by assigning priorities and using an interrupt controller with a priority scheme. When several interrupts arrive, the CPU compares their priorities: a higher-priority interrupt can preempt (interrupt) the servicing of a lower-priority one, while a lower-priority request waits until the current handler finishes. In the scenario, the hardware-failure NMI (b) has the highest priority and is serviced immediately, preempting the normal booking/upgrade interrupt (a); the upgrade message is held pending and handled after the critical condition is dealt with. This is typically implemented by the interrupt controller asserting the highest-priority request and the CPU saving the current context before branching to the appropriate interrupt service routine.

Unofficial marking guide:

• 1 mark: identifying message (b) as the non-maskable interrupt
• 1 mark: stating that an NMI cannot be masked/disabled by the processor
• 1 mark: linking message (b) to a critical hardware-failure condition requiring immediate action
• 1 mark: explaining that message (a) is a normal/deferrable maskable interrupt
• 1 mark: defining maskable interrupt as one that can be disabled/delayed
• 1 mark: defining non-maskable interrupt as one that cannot be disabled and is for urgent events
• 1 mark: stating that multiple interrupts are handled by priority assignment
• 1 mark: explaining that a higher-priority interrupt can preempt a lower-priority one
• 1 mark: applying priority to the scenario (hardware-failure NMI serviced first, upgrade interrupt deferred)
• 1 mark: mentioning the interrupt controller / saving context / ISR as the handling mechanism

Model confidence: high.

Model note: Figure 1 is represented only by its extracted text; the labels (a) and (b) are assumed to correspond to the upgrade message and the hardware-failure message respectively, as shown in the question text. The answer is an unofficial model answer, not an official marking scheme.

QUESTION A6 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (20).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The CPU is the part of the computer that fetches, decodes and executes instructions, and it is conventionally described in terms of the following components:

• Control Unit (CU): directs and coordinates the operation of the whole CPU and the rest of the computer. It decodes each instruction held in the Instruction Register, generates the timing and control signals that select the required functional units, and sequences the machine cycle (fetch, decode, execute, and where relevant memory read/write or interrupt handling). It does not itself process data; it tells the other units what to do and when.

• Arithmetic Logic Unit (ALU): performs the actual data processing — arithmetic operations (add, subtract, multiply, divide, increment/decrement) and logical operations (AND, OR, NOT, XOR, shift, compare). It takes its operands from registers, produces a result that is written back to a register, and sets condition/status flags (zero, carry, sign, overflow) in the status register.

• Registers: small, very fast storage inside the CPU that hold the data and addresses currently in use. Typical registers include the Program Counter (PC), which holds the address of the next instruction; the Instruction Register (IR), which holds the instruction being executed; the Memory Address Register (MAR), which holds the address to be accessed; the Memory Data/Buffer Register (MDR/MBR), which holds data read from or to be written to memory; the Accumulator (AC) or general-purpose registers for operands and results; and the Status/Flag register. Registers remove the need to go to memory for every intermediate value and so speed up execution.

• Internal buses / data paths: the internal connections (and the system bus interface) that carry data, addresses and control signals between the CU, ALU, registers and memory, allowing operands and results to move between components.

• Clock: generates the regular pulses that synchronise the steps of the machine cycle; the CU uses it to time each micro-operation.

(b) The bus system (address bus, data bus and control bus) is the only path over which the CPU can

© Hak Cipta Universiti Teknologi MARA

exchange information with memory and I/O. If it is faulty, the machine cycle breaks down at the fetch stage: the PC supplies the address, but a faulty address bus means the wrong location (or no location) is selected; a faulty data bus means the instruction word arriving in the IR is corrupted or absent; and a faulty control bus means the memory-read/write or timing signals are not delivered, so memory does not respond. The CU therefore cannot obtain a valid instruction to decode, so decoding and execution cannot proceed correctly — the CPU may hang, fetch garbage and execute invalid operations, or repeatedly fail/retry the fetch. In short, the machine cycle cannot complete normally because every stage depends on the bus to move instructions, operands and results between the CPU and memory.

Unofficial marking guide:

• 1 mark: identifies the Control Unit and states that it coordinates/sequences operations and decodes instructions and issues control signals

• 1 mark: identifies the ALU and states that it performs arithmetic and logical operations

• 1 mark: identifies registers and gives their function (fast internal storage for data/addresses/instructions)

• 1 mark: names at least two specific registers (e.g. PC, IR, MAR, MDR, accumulator) with correct roles

• 1 mark: identifies internal buses/data paths and their role in moving data/addresses/control signals between components

• 1 mark: identifies the clock and its role in synchronising/timing the machine cycle

• 1 mark: explains that the components work together under CU control to fetch, decode and execute instructions (overall CPU function)

• 1 mark: accurate, well-organised discussion covering all main components (quality/coverage)

• 1 mark: (b) explains that a faulty bus prevents correct fetching of instructions/operands (address, data or control signals corrupted or not delivered)

• 1 mark: (b) explains the consequence — the machine cycle cannot complete/decode/execute correctly, so the CPU hangs, executes garbage or fails

Model confidence: high.

Model note: Question text has a stray apostrophe in (a); treated as a typo. The mark split (8 + 2) is taken from the subpart labels. Answer assumes the standard von Neumann CPU component set (CU, ALU, registers, internal buses, clock); if the course lists a different fixed set of components, map the points accordingly.

QUESTION A7 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (10).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Problem with polling I/O [3 marks]

Polling (programmed I/O) requires the CPU to repeatedly read a device's status register in a busy-wait loop until the device is ready, then transfer the data itself. The main problems are: (i) the CPU wastes many instruction cycles checking status instead of doing useful work, so system throughput falls; (ii) the CPU is tied to the speed of the slow device — a slow device holds the CPU in the loop for a long time; (iii) polling must be repeated for every device, so with several devices the overhead multiplies and the CPU cannot easily serve other work; (iv) it is inefficient when transfers are infrequent or unpredictable, because the CPU may poll long before data arrives. In short, polling consumes CPU time that could otherwise be used for computation.

(b) How buffering improves overall device performance [3 marks]

Buffering uses a temporary memory area (buffer) between the CPU and the device to absorb speed and timing differences. Benefits: (i) it decouples the fast CPU from a slow peripheral, so the CPU can hand over data and continue without waiting for the device to finish; (ii) it smooths bursts — data produced faster than it can be consumed (or vice versa) is stored and drained at the device's rate, avoiding lost data and idle time; (iii) it allows block transfers, so the device and CPU can operate more independently and overlap computation with I/O, raising effective throughput and reducing the number of interrupts/status checks per byte. Overall, buffering hides device latency and mismatched transfer rates, improving utilisation and response time.

(c) How DMA improves I/O transfer performance versus polling and interrupt I/O [4 marks]

DMA (Direct Memory Access) uses a dedicated DMA controller to move blocks of data directly between memory and the device without the CPU copying each byte. The CPU programs the controller (source, destination, count) and is then free to execute other instructions while the DMA controller performs the transfer, stealing memory cycles only as needed. Compared with polling, the CPU no longer busy-waits on status, eliminating wasted cycles. Compared with interrupt-driven I/O, DMA avoids one interrupt per byte/word: only one interrupt is raised at the end of the whole block, greatly reducing interrupt overhead

© Hak Cipta Universiti Teknologi MARA

and context-switch cost. This gives high throughput for large block transfers (disk, network, graphics), better CPU utilisation through overlap of I/O and computation, and fewer status/interrupt handling operations. The trade-off is added hardware (the DMA controller) and some memory-cycle contention, but for bulk transfers DMA is markedly faster and more efficient than either polling or per-character interrupt I/O.

Unofficial marking guide:

• 1 mark: states polling requires the CPU to repeatedly check the device status in a busy-wait loop
• 1 mark: explains CPU time/cycles are wasted waiting, reducing throughput and tying the CPU to the slow device
• 1 mark: notes inefficiency with multiple devices or infrequent/unpredictable transfers (overhead multiplies)
• 1 mark: defines buffering as a temporary memory area between CPU and device
• 1 mark: explains decoupling of fast CPU and slow device / absorbing speed mismatch
• 1 mark: explains smoothing of bursts and/or block transfer enabling overlap, raising throughput and reducing interrupts
• 1 mark: explains DMA controller transfers blocks directly between memory and device without CPU copying each byte
• 1 mark: explains CPU is freed to do other work / overlap of I/O with computation
• 1 mark: contrasts with polling (no busy-wait) and interrupt I/O (one interrupt per block instead of per byte)
• 1 mark: notes high throughput for large block transfers and trade-off (extra hardware/memory-cycle contention)
Model confidence: high.
Model note: OCR '1/0' read as 'I/O'. Question total is 10 marks; marking allocates 3+3+4.

QUESTION A8 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (5).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Direct mapping does not need a replacement algorithm. In direct mapping each main-memory block maps to exactly one fixed cache line (line = block address mod number of lines), so when a new block must be cached there is only one possible line it can occupy. That line is simply overwritten (its valid bit set and tag updated); no choice between candidate lines arises, hence no replacement policy is required. (By contrast, fully associative and set-associative caches have several candidate lines per block and therefore need a replacement algorithm.)

(b) Four types of replacement algorithm:

1. Random replacement: a line is chosen at random from the candidate set (or from the whole cache in a fully associative cache) and evicted. It is very simple and cheap to implement in hardware, requires no bookkeeping of usage, and avoids pathological worst cases, but it takes no account of locality so hit rate is generally poorer than usage-based schemes.

2. FIFO (First-In, First-Out): lines are evicted in the order they were loaded, maintained by a circular pointer/queue per set. It is simple and cheap, but it ignores how recently or how often a line has been used, so it may evict a heavily used block that was loaded early (a form of Belady-type anomaly).

3. LRU (Least Recently Used): the line that has gone longest without being referenced is evicted, exploiting temporal locality. It gives good hit rates but requires tracking of access order (e.g. counters or a stack), which is costly in hardware and becomes impractical for large associativity, so approximations are often used.

4. LFU (Least Frequently Used): the line with the smallest reference count is evicted, using a counter per line. It favours frequently used blocks, but counters must be maintained and periodically reset, and blocks that were heavily used in the past but are no longer needed can linger (poor adaptation to phase changes).

(Other acceptable schemes, e.g. NRU/clock approximations, may be credited as alternatives.)

Unofficial marking guide:

• 2 marks: (a) correctly identifies direct mapping as needing no replacement algorithm
• 2 marks: (b) first replacement algorithm correctly named and described
• 2 marks: (b) second replacement algorithm correctly named and described
• 2 marks: (b) third replacement algorithm correctly named and described
• 2 marks: (b) fourth replacement algorithm correctly named and described
Model confidence: high.
Model note: Question stem says 'flush to main memory'; in a write-back cache eviction may require a write-back, but this does not

affect the answer. Any four valid replacement algorithms (e.g. Random, FIFO, LRU, LFU, NRU/clock) are acceptable for part (b).

QUESTION A9 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (15).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The phases in the machine cycle are: (1) Fetch — the next instruction is fetched from main memory into the CPU; (2) Decode — the instruction is decoded to determine the operation and operands; (3) Execute — the operation is carried out by the ALU/control unit; (4) Store (Write-back) — the result is written back to a register or to main memory. (Some texts split fetch into fetch and increment-PC, or add an interrupt/acknowledge phase; the four-phase fetch–decode–execute–store cycle is the standard answer.)

(b) The CPU components and their functions in the machine cycle are: the Program Counter (PC) holds the address of the next instruction to be fetched and is incremented during fetch; the Memory Address Register (MAR) receives the address of the memory location to be accessed and passes it to the address bus; the Memory Data Register (MDR) buffers the instruction or data read from (or written to) main memory; the Instruction Register (IR) holds the instruction currently being decoded/executed; the Control Unit (CU) decodes the instruction and generates the timing and control signals that sequence the fetch, decode, execute and store phases; the Arithmetic Logic Unit (ALU) performs arithmetic and logical operations during execute; and the general-purpose registers/accumulator hold operands and results. The internal buses carry data and control signals between these units.

(c) If access to main memory is blocked, the fetch phase cannot complete: the PC's address cannot be read, so no instruction reaches the IR and the decode/execute/store phases cannot proceed. The CPU stalls (waits, inserting wait states) or, if a memory fault/interrupt is signalled, traps to an error/interrupt handler. In effect the machine cycle halts at fetch and the program cannot make progress until memory access is restored.

Unofficial marking guide:

- *1 mark: Fetch phase correctly named*
- *1 mark: Decode phase correctly named*
- *1 mark: Execute phase correctly named*
- *1 mark: Store/write-back phase correctly named (4 marks total for (a))*
- *1 mark: PC and MAR roles in fetch (address of next instruction / memory address)*
- *1 mark: MDR and IR roles (buffering data / holding current instruction)*
- *1 mark: Control Unit decodes and generates control/timing signals*
- *1 mark: ALU performs operations and registers hold operands/results (4 marks total for (b))*
- *1 mark: fetch cannot complete / CPU stalls or waits*
- *1 mark: cycle cannot proceed to decode/execute/store, or interrupt/error trap (2 marks total for (c))*

Model confidence: high.

Model note: Question text is complete; no figure required. Some textbooks use a five-phase cycle (fetch, decode, execute, memory, write-back) or include an interrupt-acknowledge phase; the four-phase answer is accepted as standard.

QUESTION A10 (10 marks)

Topic: Data representation and number systems

a) Answer: 314510 = C4916

- *1 mark for repeated division by 16.*
- *1 mark for correct remainders.*
- *1 mark for reading remainders in reverse order.*
- *1 mark for the correct hexadecimal digits.*
- *1 mark for the base subscript.*

b) Answer: ED116 = 379310

- *1 mark for place values 16^2, 16^1 and 16^0.*
- *1 mark for correct digit values.*
- *1 mark for correct products.*
- *1 mark for correct addition.*
- *1 mark for the final decimal value.*

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (17).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 2 reads y = A(A + 2B/C-D 2). The only consistent reading with the available instructions (no exponentiation) is y = A × (A + 2B/C − D²), i.e. the trailing 2 is a superscript square on D. This is assumed throughout; if the figure shows a different expression, the operand order changes but the method is identical.

Sub-expression plan (common to all three programs): compute T1 = 2B/C, T2 = D×D, T3 = A + T1 − T2, then y = A × T3. A, B, C, D are read-only; x, y, z are temporaries; final result in y.

Zero-address program (stack machine):

```
PUSH A        ; stack: A
    PUSH B        ; stack: A, B
    PUSH B        ; stack: A, B, B
    ADD           ; stack: A, 2B       (B+B)
    PUSH C        ; stack: A, 2B, C
    DIV           ; stack: A, 2B/C
    ADD           ; stack: A + 2B/C
    PUSH D        ; stack: (A+2B/C), D
    PUSH D        ; stack: (A+2B/C), D, D
    MUL           ; stack: (A+2B/C), D*D
    SUB           ; stack: A + 2B/C - D*D
    PUSH A        ; stack: (A+2B/C-D*D), A
    MUL           ; stack: A*(A+2B/C-D*D)
    POP y         ; store final answer in y
```

One-address program (accumulator machine):

```
LOAD B        ; ACC = B
    ADD B         ; ACC = 2B
    DIV C         ; ACC = 2B/C
    ADD A         ; ACC = A + 2B/C
    STOR x        ; x = A + 2B/C   (save partial result)
    LOAD D        ; ACC = D
    MUL D         ; ACC = D*D
    STOR z        ; z = D*D
    LOAD x        ; ACC = A + 2B/C
    SUB z         ; ACC = A + 2B/C - D*D
    MUL A         ; ACC = A*(A + 2B/C - D*D)
    STOR y        ; y = final answer
```

Three-address program (register/operand form ADD(p,q,r) meaning p = q op r):

```
ADD (x, B, B)     ; x = 2B
    DIV (x, x, C)     ; x = 2B/C
    ADD (x, A, x)     ; x = A + 2B/C
    MUL (z, D, D)     ; z = D*D
    SUB (x, x, z)     ; x = A + 2B/C - D*D
    MUL (y, A, x)     ; y = A*(A + 2B/C - D*D)  final answer in y
```

Comment on correctness: In every version the read-only registers A, B, C, D are only ever read as source operands, the temporaries x and z hold intermediate values, and the final result is written to y, satisfying the stated constraints.

Unofficial marking guide:

• 4 marks: zero-address program — correct use of PUSH/POP with stack semantics, correct operand order for DIV and SUB, and final POP into y

• 4 marks: one-address program — correct LOAD/STOR/ADD/SUB/MUL/DIV sequence, correct use of temporaries x and z, final STOR y

© Hak Cipta Universiti Teknologi MARA

• 4 marks: three-address program — correct ADD/DIV/MUL/SUB with (p,q,r) destination-first convention, correct final MUL into y

*• 4 marks: correct evaluation of the expression (2B/C, D*D, A + 2B/C − D*D, then multiply by A) in all three programs*

• 2 marks: comments and correct assembly notation throughout, with A, B, C, D unmodified and final answer in y

• 2 marks: overall clarity, consistent register/temporary usage and no redundant or missing operations

Model confidence: medium.

Model note: The equation in Figure 2 is OCR-ambiguous ('D 2'); assumed to be D squared since no exponent instruction exists. The three-address table shows 'MUL (p.q.r)' — read as MUL(p,q,r). The (p,q,r) convention is assumed to be destination-first (p = q op r); if the lecturer uses source-first ordering, swap the operand positions accordingly. Figure 2 itself was not available beyond the extracted text.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA